Module 9: Assignment-Working with Raw SQL in pgAdmin

Docker Compose Setup:

```
version: '3.8'

services:
  # Service: web
  web:
    build: .
    container_name: fastapi_calculator
    ports:
      - "8000:8000"
    volumes:
      - .:/app
    environment:
      PYTHONDONTWRITEBYTECODE: 1
      PYTHONUNBUFFERED: 1
      DATABASE_URL: postgresql://postgres:postgres@db:5432/fastapi_db
    command: uvicorn main:app --host 0.0.0.0 --port 8000 --reload
    depends_on:
      - db
    networks:
      - app-network

  # Service: db (PostgreSQL)
  db:
    image: postgres:15
    container_name: postgres_db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: fastapi_db
    ports:
```

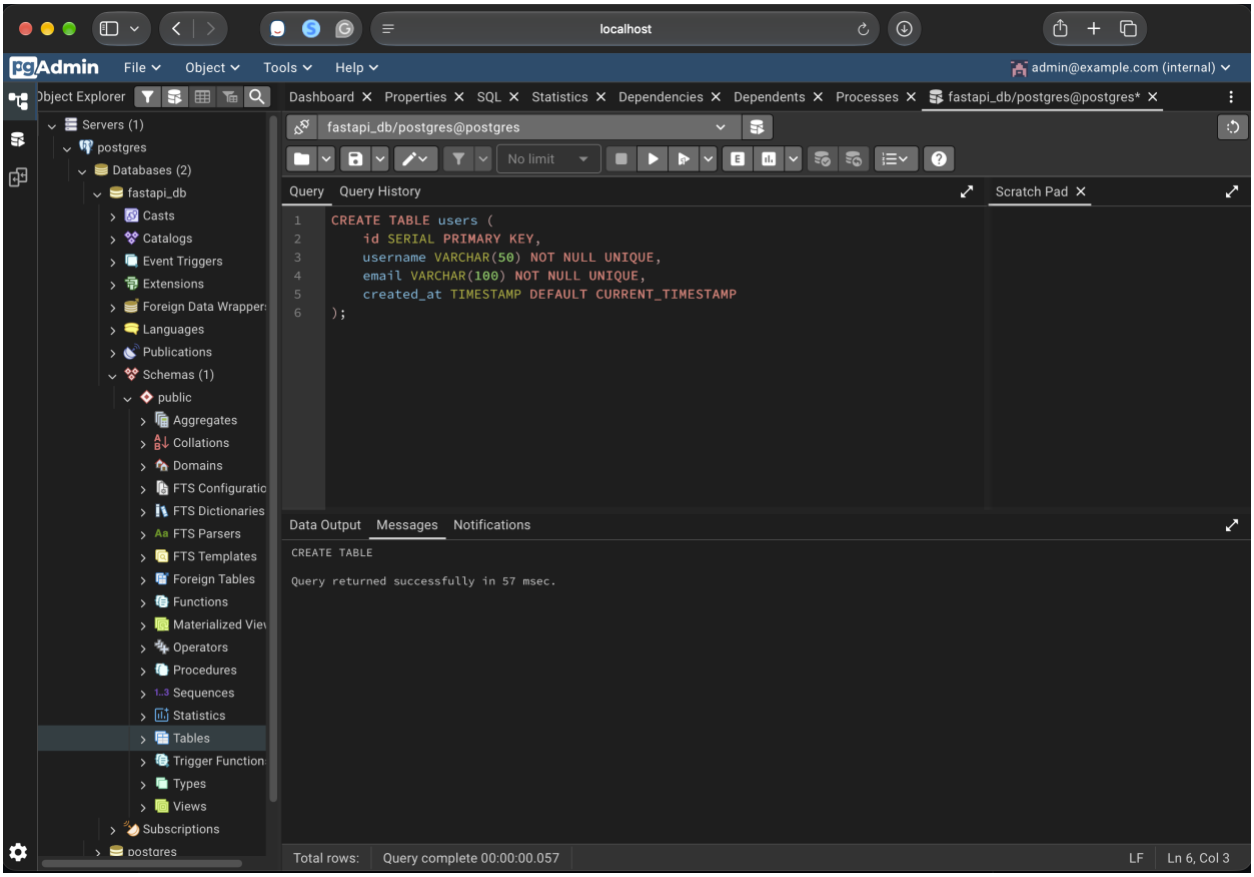
```
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    - app-network

# Service: pgadmin
pgadmin:
  image: dpage/pgadmin4  # Updated to a newer version
  container_name: pgadmin
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@example.com
    PGADMIN_DEFAULT_PASSWORD: admin
  ports:
    - "5050:80"  # Ensure port 5050 is free or change if necessary
  volumes:
    - pgadmin_data:/var/lib/pgadmin
  depends_on:
    db:
      condition: service_healthy
  networks:
    - app-network

volumes:
  postgres_data:
    driver: local
  pgadmin_data:
    driver: local

networks:
  app-network:
    driver: bridge
```

SQL command executions in pgAdmin showing successful table creation, data insertion, queries, updates, and deletions.



(A) Create Tables:

Table Creation: ‘users’ table

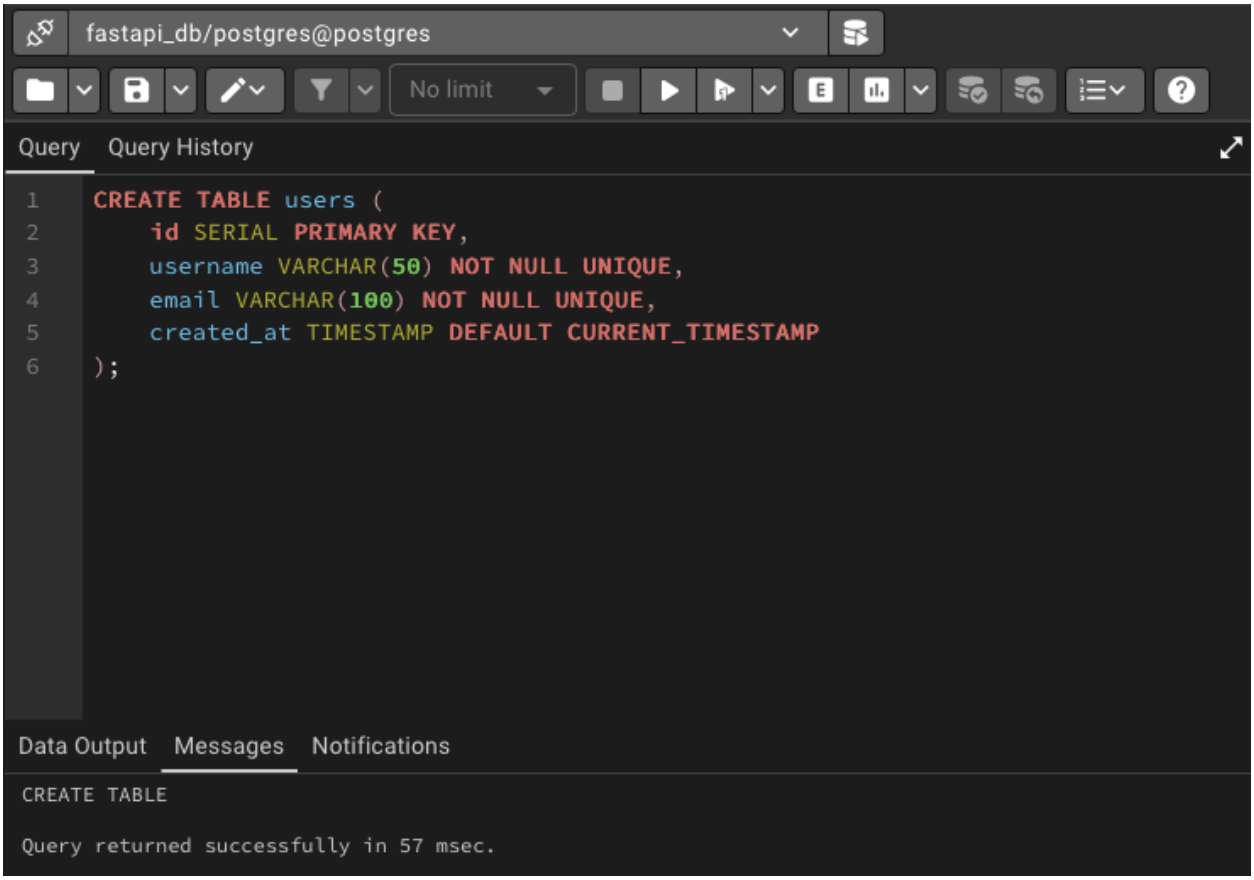
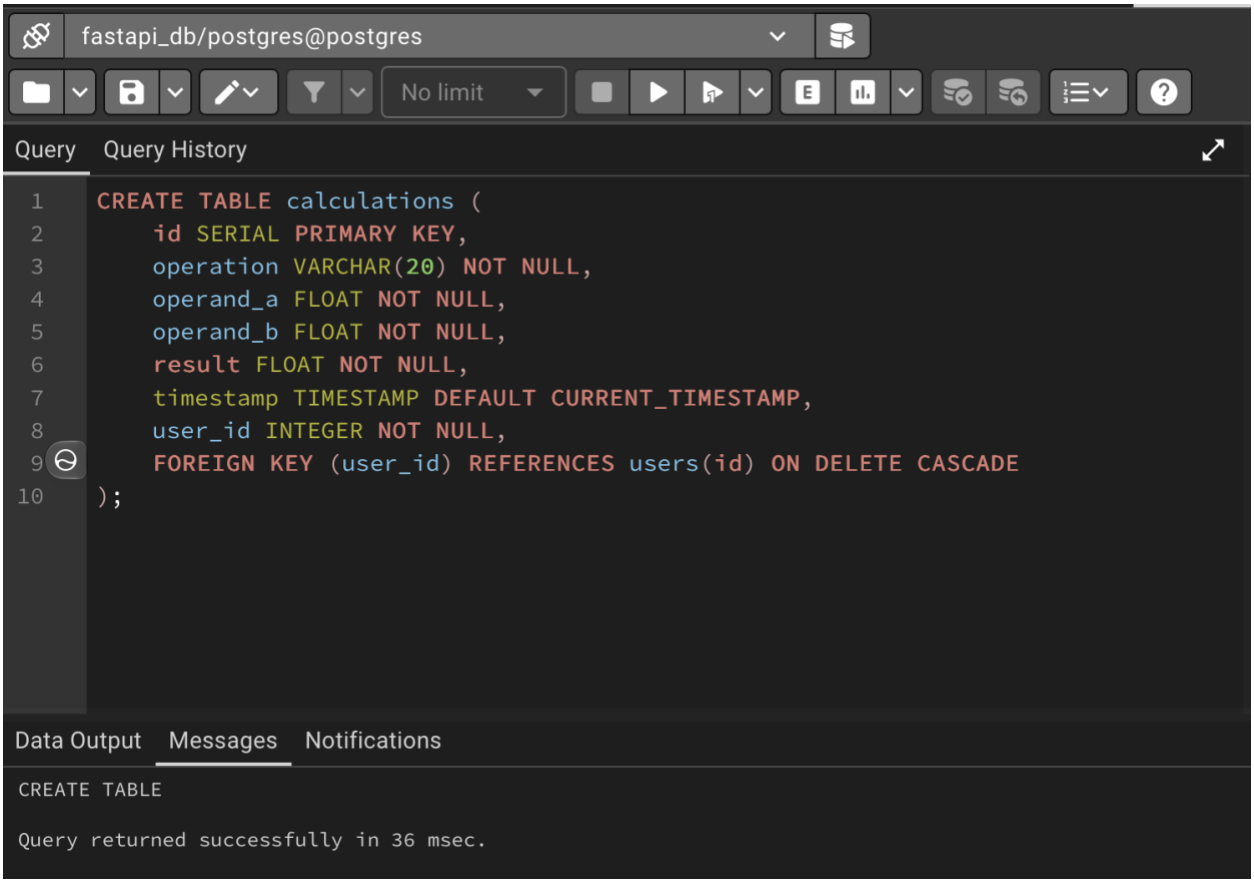
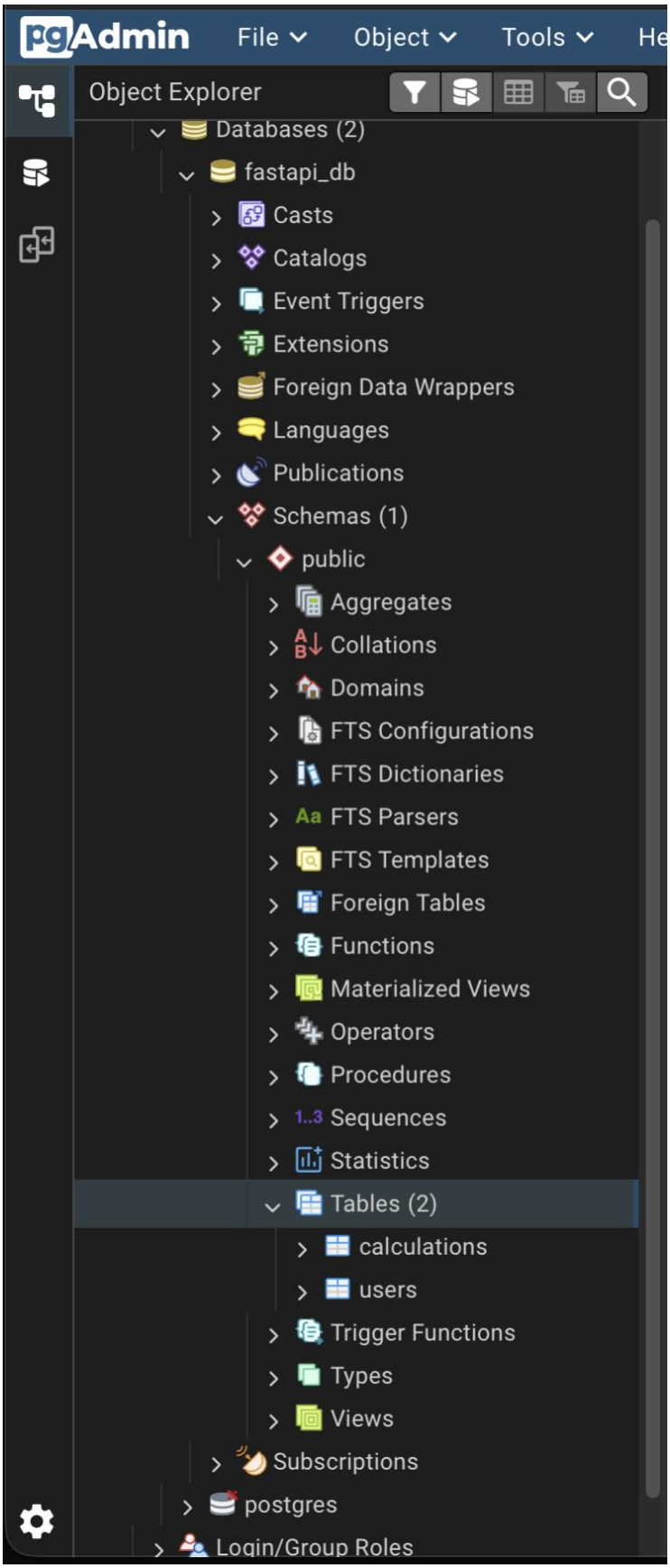


Table Creation: ‘calculations’ table

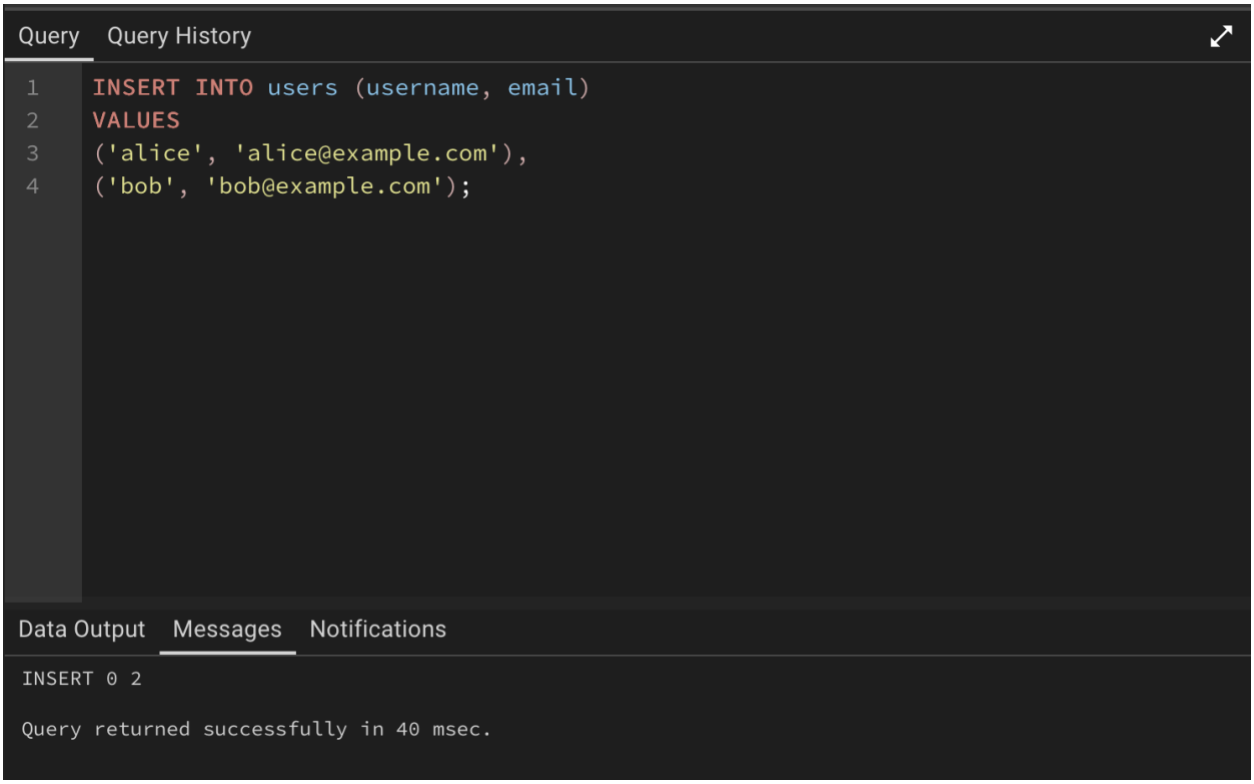


Showing Tables calculations and users that are created.

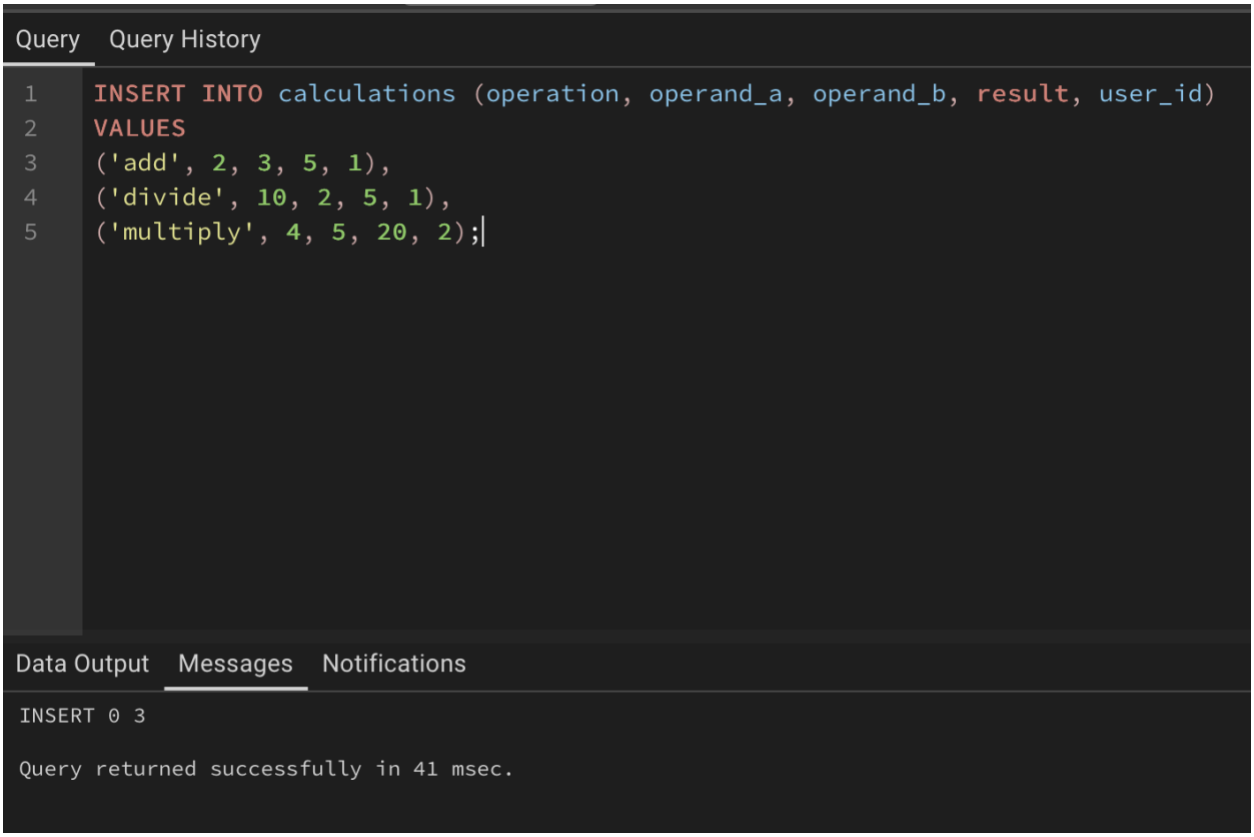

pgAdmin
File
Object
Tools
He
Object Explorer
Databases (2)
fastapi_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Statistics
Tables (2)
calculations
users
Trigger Functions
Types
Views
Subscriptions
postgres
Login/Group Roles

(B) Insert Records

Inserting Records into ‘users’ table

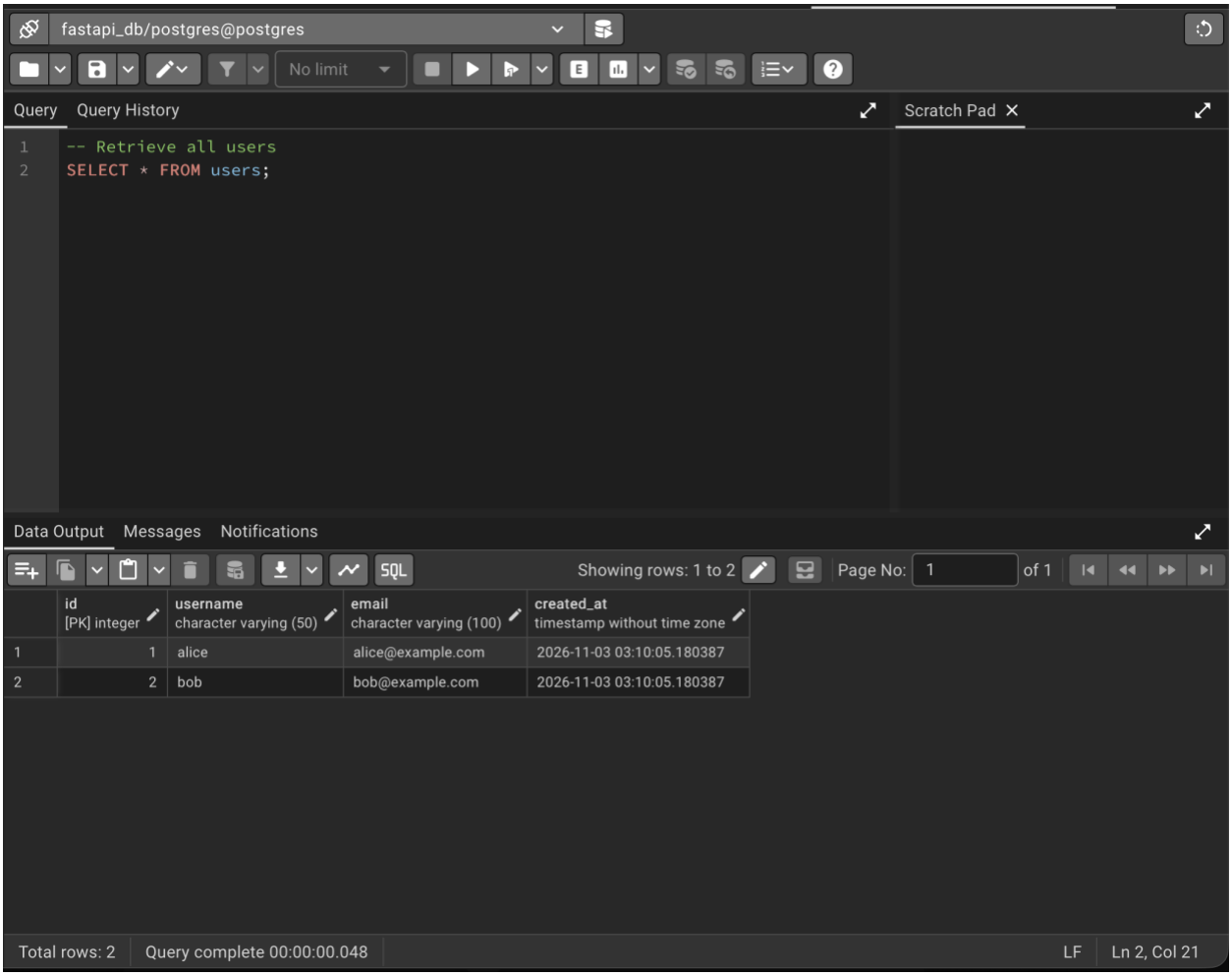


Inserting Records into ‘calculations’ table



(C) Query Data

Retrieve all the users



Retrieving all calculations that are in the table

Query | Query History | Scratch Pad

```
-- Retrieve all calculations
SELECT * FROM calculations;
```

Data Output | Messages | Notifications

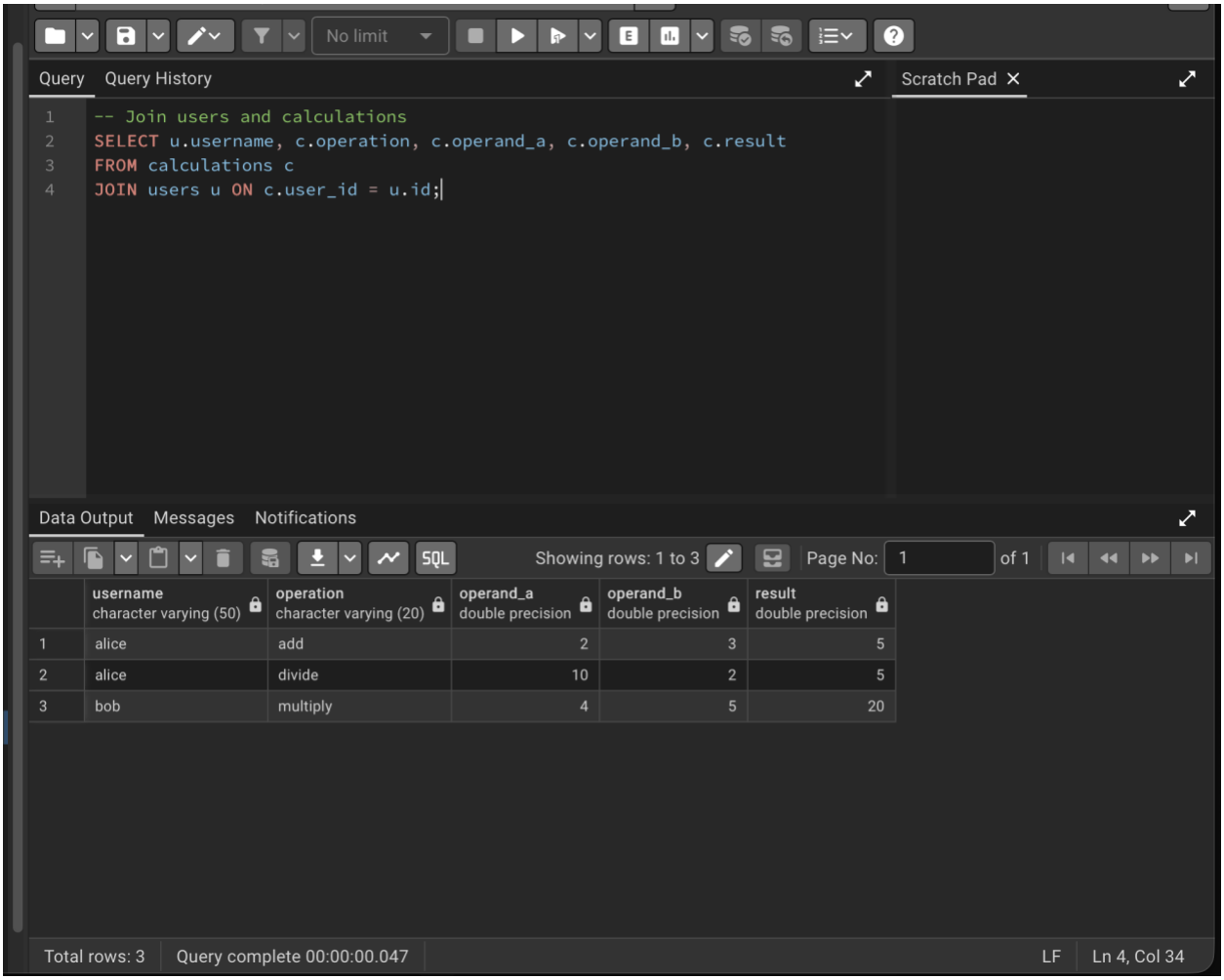
Showing rows: 1 to 3 | Page No: 1 of 1

	id [PK] integer	operation character varying (20)	operand_a double precision	operand_b double precision	result double precision	timestamp timestamp without time zone	user_id integer
1	1	add	2	3	5	2026-11-03 03:11:45.809959	1
2	2	divide	10	2	5	2026-11-03 03:11:45.809959	1
3	3	multiply	4	5	20	2026-11-03 03:11:45.809959	2

Total rows: 3 | Query complete 00:00:00.060 | LF | Ln 2, Col 28

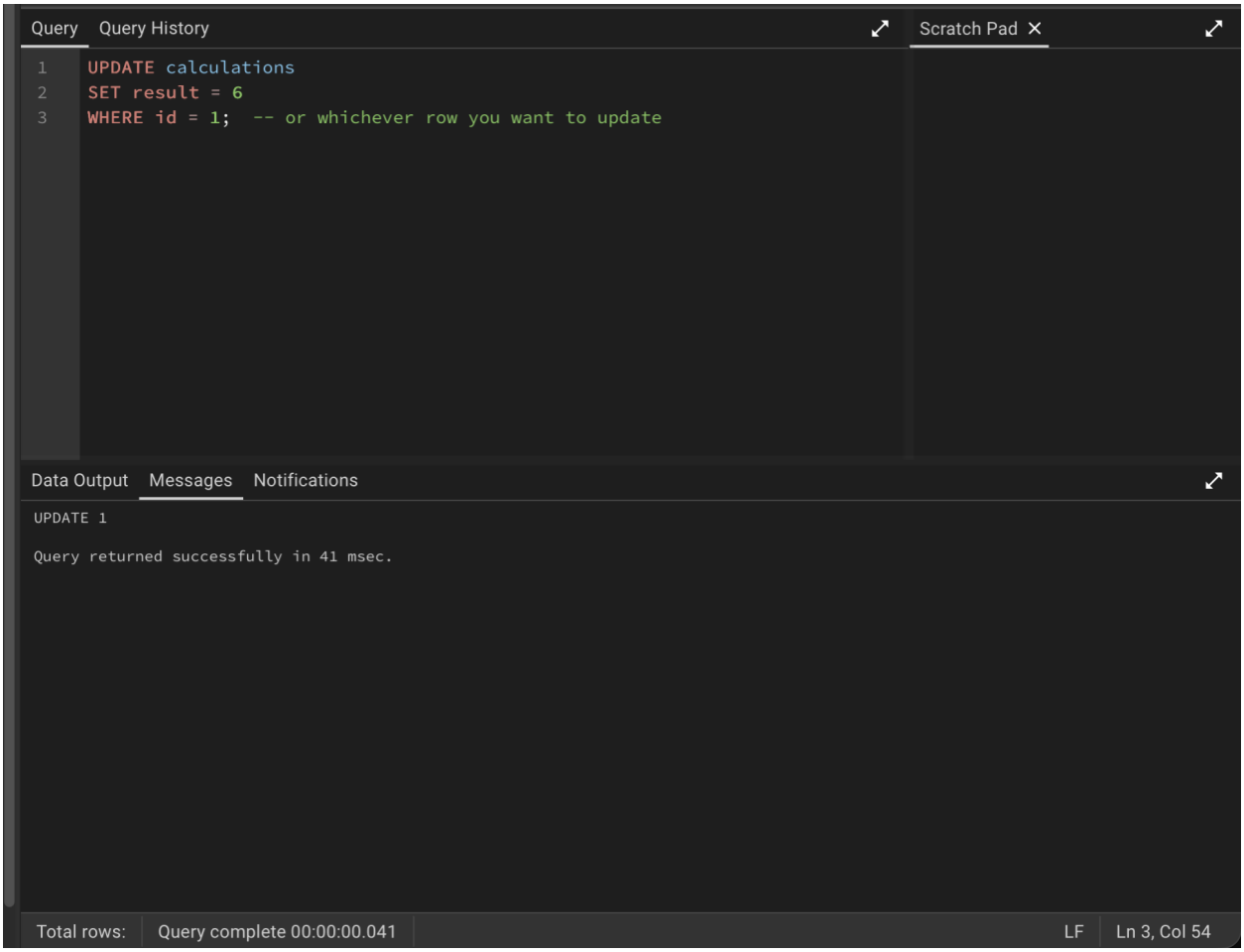
Join users and calculations table and retrieving selected columns from the

respective tables



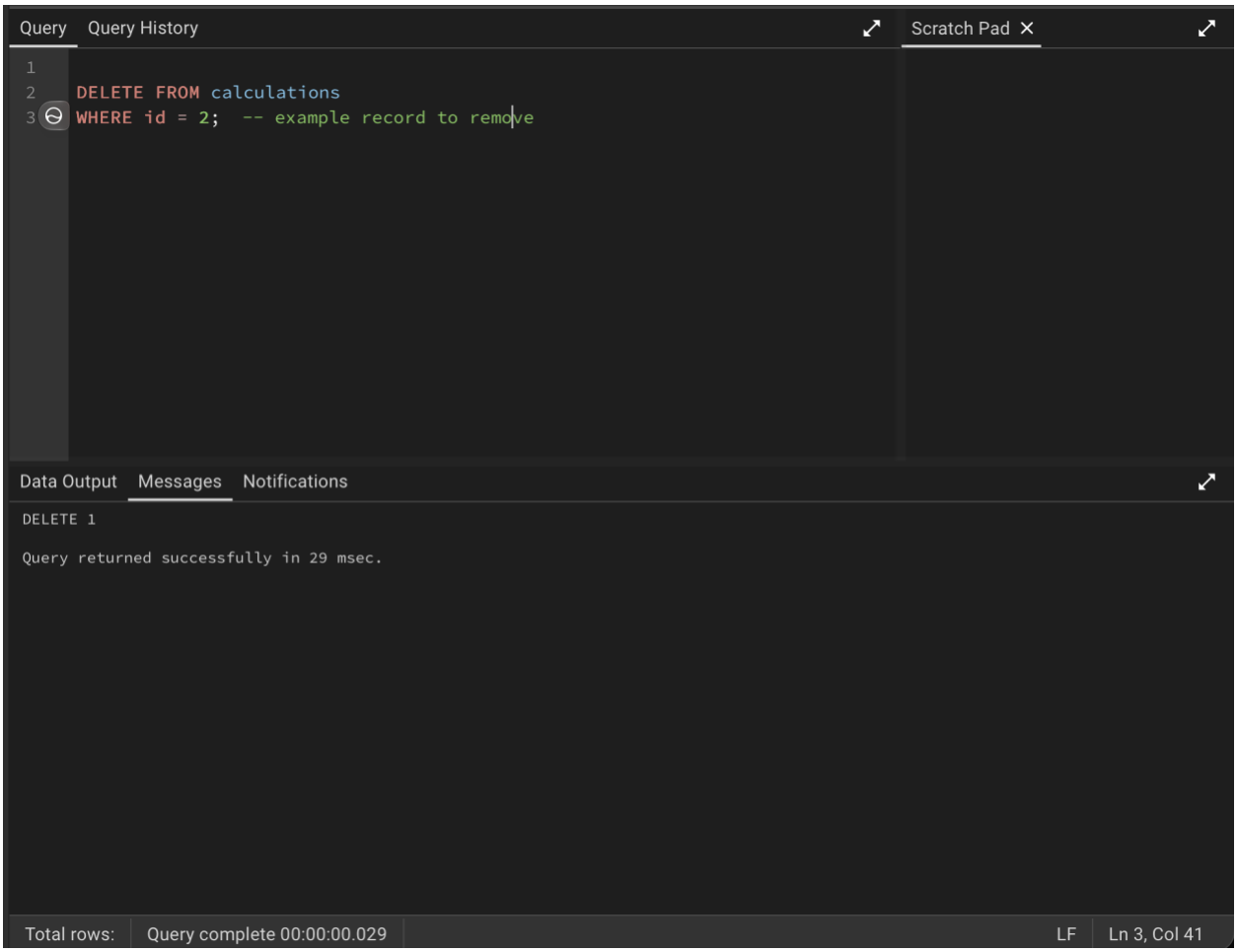
(D) Update a Record

Updating table ‘calculations’ for id - 1 with result as *6*

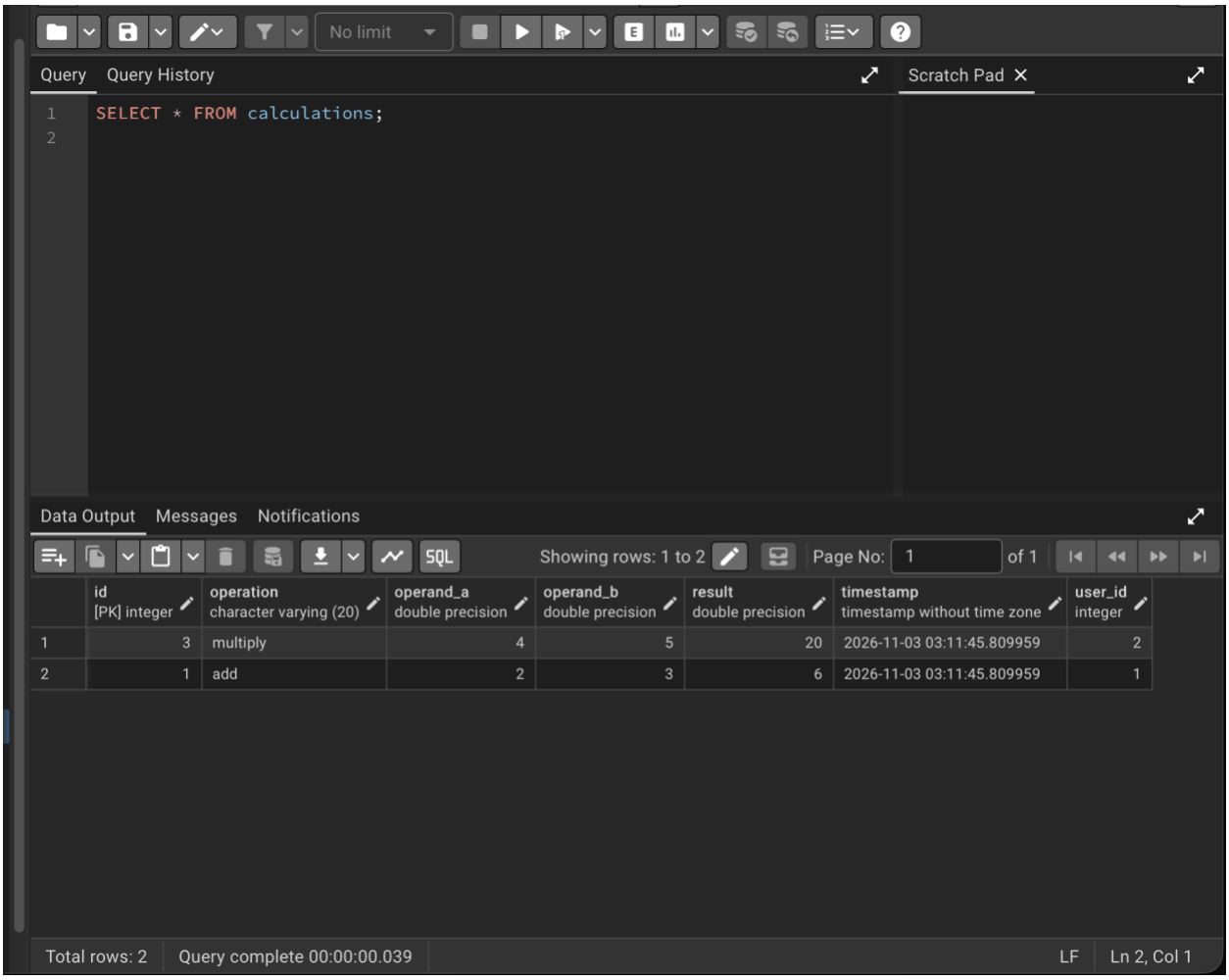


(E) Delete a Record

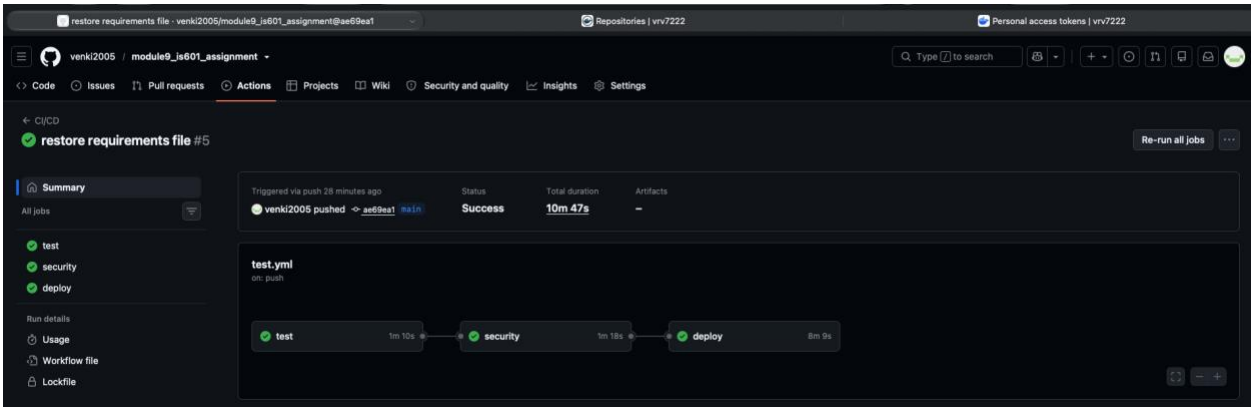
Deleting a row from ‘calculations’ table for id -> 2



Data in ‘calculations’ table after we deleted the row with id – 2 and u[dating the result as 6 for id - 1



Successful run of Docker Compose services.



docker-compose.yml

```yaml
You, 1 hour ago | author (You)
version: '3.8'

Run All Services
services:
  # Service: web
  Run Service
  web:
    build: .
    container_name: fastapi_calculator
    ports:
      - "8000:8000"
    volumes:
      - .:/app
    environment:
      PYTHONDONTWRITEBYTECODE: 1
      PYTHONUNBUFFERED: 1      You, 1 hour ago • adding all the files for module 9 assignment
      DATABASE_URL: postgresql://postgres:postgres@db:5432/fastapi_db
    command: uvicorn main:app --host 0.0.0.0 --port 8000 --reload
    depends_on:
      - db
    networks:
      - app-network

  # Service: db (PostgreSQL)
  Run Service
  db:
    image: postgres:15
    container_name: postgres_db
    environment:
```

Problems 27 Output Debug Console Terminal Ports

zsh

```
(venv) (base) venkatareddyvatrapu@MacBook-Pro module9_is601_assignment % docker compose ps
WARN[0000] /Users/venkatareddyvatrapu/NJIT-MS-Course/Python/module9_is601_assignment/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                          COMMAND                  SERVICE   CREATED       STATUS                    PORTS
fastapi_calculator   module9_is601_assignment-web   "uvicorn main:app --…"   web       2 hours ago   Up 2 hours (unhealthy)   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
pgadmin             dpage/pgadmin4                 "/entrypoint.sh"         pgadmin   2 hours ago   Up 2 hours               0.0.0.0:5050->80/tcp, [::]:5050->80/tcp
postgres_db         postgres:15                    "docker-entrypoint.s…"   db        2 hours ago   Up 2 hours (healthy)     0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
(venv) (base) venkatareddyvatrapu@MacBook-Pro module9_is601_assignment %
```

You, 1 hour ago VenkataVatrapu (1 hour ago) Ln 14, Col 26 Spaces: 2 UTF-8 LF

requirements.txt Dockerfile test_e2e.py Hello World & Calculator Demo docker-compose.yml test.yml 3 .trivyignore 9+

http://127.0.0.1:8000/

Hello World

Calculator

3 4

Add Subtract Multiply Divide

Result: 0.75

Assignment Reflection

This assignment gave me valuable hands-on experience with PostgreSQL and built on my previous experience with Microsoft SQL Server, MySQL, MongoDB, and Oracle SQL. Following the steps in the lecture video made configuring PostgreSQL, FastAPI, and pgAdmin within Docker straightforward. Initially, I encountered challenges with the Docker image tag, an error indicating that a container name was already in use, and synchronization during end-to-end (E2E) testing. Working through these issues gave me practical troubleshooting experience and helped me feel more comfortable with the setup. Overall, this assignment was a rewarding learning experience, and I finished feeling more confident working with PostgreSQL and the supporting tools.